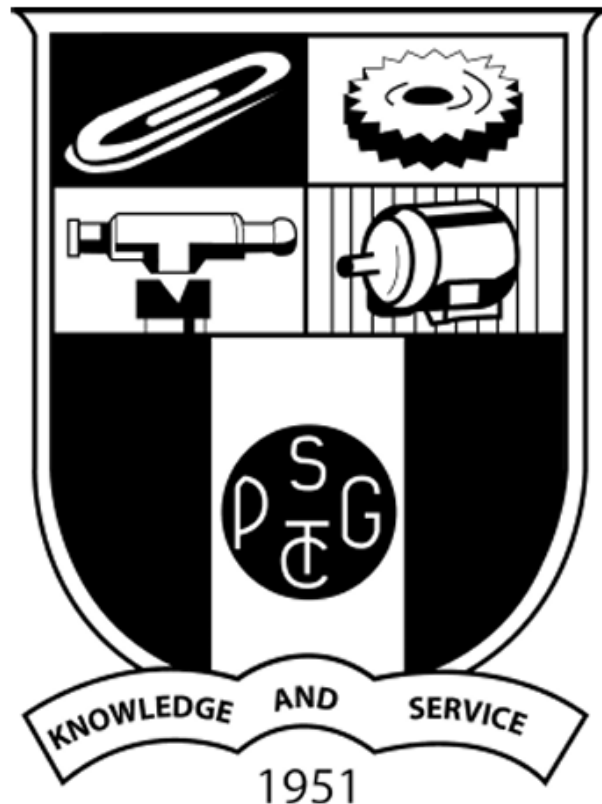


PSG COLLEGE OF TECHNOLOGY

PEELAMEDU, COIMBATORE – 641004



23I412-EMBEDDED SYSTEM DESIGN LABORATORY

MINI PROJECT REPORT

**Smart Water Quality Monitoring System Using IoT-Based Embedded
Platforms: Simulation and Hardware Implementation**

TEAM MEMBERS:

24I343 - PRIYADHARSHINI T S

24I345 - NETHRA R

24I358 - SOWMYA P NAGARAJAN

24I374 – VINOTHINI G

Table of Contents

Abstract	3
I. Introduction	4
II. Problem Statement	4
III. Proposed System	4
IV. Simulation — Raspberry Pi Pico on Wokwi	5
V. Hardware Implementation — Arduino UNO	7
VI. System Classification and Alert Logic	11
VII. Applications	11
VIII. Comparative Advantages Over Conventional Low-Cost Checkers	12
IX. Key Features and Advantages	12
X. Future Enhancements	13
XI. Conclusion	13
References	14

Smart Water Quality Monitoring System Using IoT-Based Embedded Platforms: Simulation and Hardware Implementation

Abstract

Access to clean and safe water is a fundamental necessity for human health and well-being. Conventional manual water quality testing methods are time-consuming, costly, and lack the capability for real-time continuous monitoring. This paper presents the design and implementation of a Smart Water Quality Monitoring System that monitors critical water parameters — pH and turbidity — in real time using embedded microcontroller platforms. The system was developed in two stages: first as a simulation using the Raspberry Pi Pico on the Wokwi simulator with MicroPython, and subsequently implemented as a physical hardware prototype using the Arduino UNO with actual sensors (REES52 pH sensor and JD23-2 turbidity sensor). The transition from Raspberry Pi Pico to Arduino UNO was necessitated by hardware compatibility issues encountered when attempting to interface the pH module with the Raspberry Pi — the analog pH sensor could not be reliably connected to the Raspberry Pi's GPIO, leading to the adoption of the Arduino UNO which natively supports analog input. The system classifies water quality into three states — SAFE, WARNING, and UNSAFE — and provides visual and audio alerts through tri-color LEDs and a buzzer, along with diagnostic recovery suggestions output via serial monitor. Results demonstrate that the proposed system provides a low-cost, automated, and scalable solution for water quality monitoring suitable for deployment in residential, agricultural, and industrial environments.

Keywords: *Water quality monitoring, IoT, Arduino UNO, Raspberry Pi Pico, pH sensor, turbidity sensor, embedded systems, real-time monitoring.*

I. Introduction

Water contamination is a growing global challenge. According to the World Health Organization (WHO), unsafe water is responsible for an estimated 485,000 diarrheal deaths annually, with millions more affected by waterborne diseases. Traditional water quality assessment relies on laboratory analysis, which involves significant delays and costs, making continuous or large-scale monitoring impractical for many communities.

Emerging Internet of Things (IoT) and embedded systems technologies have opened new avenues for affordable, real-time environmental monitoring. Smart sensors coupled with microcontrollers can continuously track water parameters such as pH (acidity/alkalinity) and turbidity (cloudiness due to suspended particles) — two of the most critical indicators of water potability as defined by standards such as those from WHO and the Bureau of Indian Standards (BIS).

This paper presents a Smart Water Quality Monitoring System designed to address the limitations of conventional approaches. The system reads water quality parameters in real time, classifies the water condition as SAFE, WARNING, or UNSAFE based on predefined thresholds, triggers LED and buzzer alerts, and outputs corrective action suggestions via serial communication. The project was developed and validated in two phases: simulation using a Raspberry Pi Pico on the Wokwi platform, and hardware implementation using an Arduino UNO microcontroller with physical sensors.

II. Problem Statement

The following key challenges motivate the development of this system:

- Manual testing is periodic and cannot detect sudden contamination events.
- Laboratory-based water quality testing is expensive and inaccessible in rural or resource-limited areas.
- Conventional low-cost water quality checkers provide single-parameter readings with no automated alerts or corrective guidance.
- Lack of actionable feedback means end users cannot respond promptly to unsafe conditions.
- No integration between sensing, classification, and alert mechanisms in commercially affordable devices.

III. Proposed System

A. System Overview

The Smart Water Quality Monitoring System measures two water parameters — pH and turbidity — and performs automated classification with real-time feedback. The system architecture consists of three functional layers: (1) sensing, (2) processing and classification, and (3) output and alerting.

B. Parameter Thresholds

The system applies thresholds aligned with WHO and BIS water quality standards:

Parameter	Safe Range	Warning Range	Unsafe Condition
pH	6.5 – 8.5	5.0 – 9.0	< 5.0 or > 9.0
Turbidity (NTU)	< 5 NTU	5 – 10 NTU	> 10 NTU

Table I. Water Quality Parameter Thresholds

IV. Simulation — Raspberry Pi Pico on Wokwi

A. Platform and Tools

The simulation was conducted using the Wokwi online embedded systems simulator. The Raspberry Pi Pico microcontroller was selected for its dual-core ARM Cortex-M0+ processor and built-in ADC channels. MicroPython was used as the programming language due to its simplicity and rapid prototyping capabilities.

B. Sensor Configuration

Two potentiometers were used in the simulation to emulate the analog output of a pH sensor (ADC pin GP26) and turbidity sensor (ADC pin GP28). Analog values were scaled to represent practical measurement ranges: pH (0–14) and turbidity (0–20 NTU).

C. Output Indicators

Tri-color LEDs (Green: GP2, Yellow: GP3, Red: GP4) and a buzzer (GP13) provide real-time visual and audio feedback. The system prints diagnostic information — parameter values, status, causes, correction magnitudes, and recovery actions — via serial output every 2 seconds.

D. Simulation Code

The following MicroPython code was used for the Wokwi simulation on the Raspberry Pi Pico:

Simulation Code (MicroPython — Raspberry Pi Pico / Wokwi)

```
from machine import ADC, Pin
import time

# Sensors (Potentiometers)
ph_sensor = ADC(26)
turb_sensor = ADC(28)

# Indicators
green = Pin(2, Pin.OUT)
yellow = Pin(3, Pin.OUT)
red = Pin(4, Pin.OUT)
buzzer = Pin(13, Pin.OUT)

# Read sensor values
def read_values():
    ph = ph_sensor.read_u16() * 14 / 65535
    turb = turb_sensor.read_u16() * 20 / 65535
    return ph, turb

# Analyze condition
def analyze(ph, turb):
    issues = []
    causes = []
    actions = []

    # pH check
    if ph < 6.5:
        causes.append("pH Low")
        issues.append("Increase pH by {:.2f}".format(6.5 - ph))
        actions.append("Add base (alkaline solution)")
    elif ph > 8.5:
        causes.append("pH High")
        issues.append("Decrease pH by {:.2f}".format(ph - 8.5))
        actions.append("Add acid")

    # Turbidity check
    if turb >= 5:
        causes.append("High Turbidity")
        issues.append("Reduce Turbidity by {:.2f}".format(turb - 5))
        actions.append("Start filtration / sedimentation")

    # Decide status
    if not causes:
        status = "SAFE"
    elif (5 <= ph <= 9) and turb < 10:
        status = "WARNING"
    else:
        status = "UNSAFE"

    return status, causes, issues, actions

# Main loop
while True:
```

```
ph, turb = read_values()
status, causes, issues, actions = analyze(ph, turb)
print("pH:", round(ph, 2))
print("Turbidity:", round(turb, 2))
print("Status:", status)
if causes:
    print("Cause:", " ".join(causes))
if issues:
    print("Correction:", " ".join(issues))
# Recovery actions (only suggestions)
if actions:
    print("Recovery Actions:", " ".join(actions))
print("-----")

# Indicators
green.off()
yellow.off()
red.off()
buzzer.off()

if status == "SAFE":
    green.on()
elif status == "WARNING":
    yellow.on()
else:
    red.on()
    buzzer.on()

time.sleep(2)
```

E. Simulation Diagram

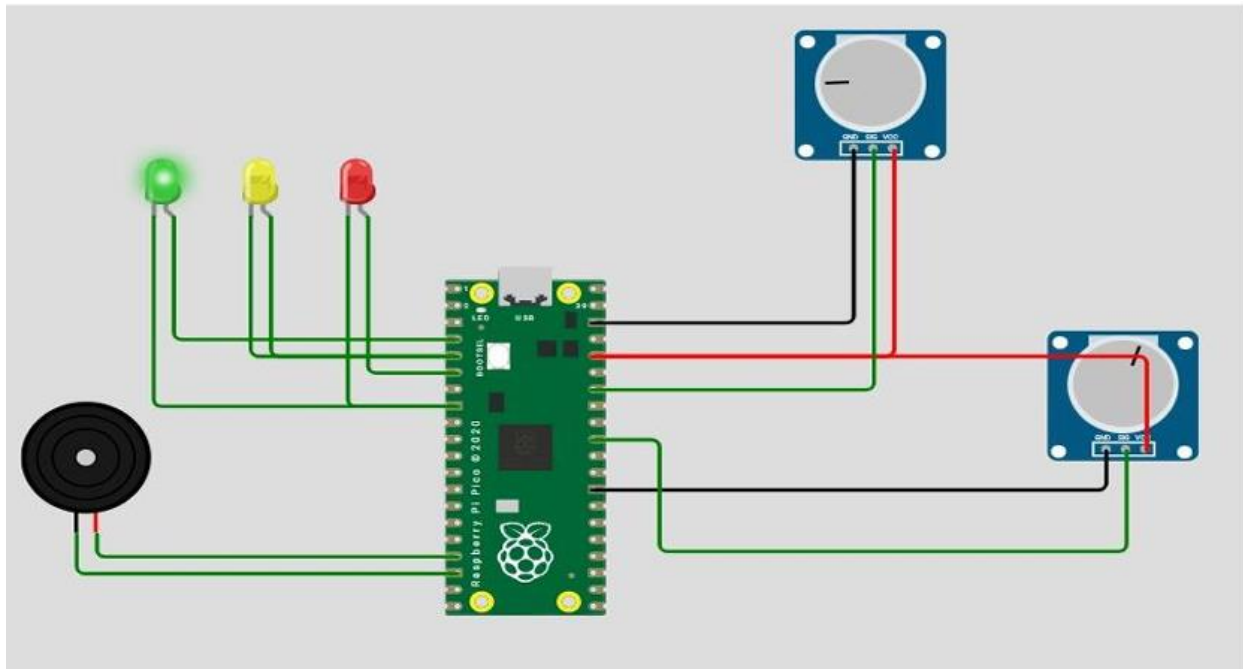


Fig. 1. Wokwi Simulation Circuit — Raspberry Pi Pico with potentiometer sensors and indicator LEDs

V. Hardware Implementation — Arduino UNO

A. Reason for Transition from Raspberry Pi Pico to Arduino UNO

During the initial hardware prototyping phase, an attempt was made to interface the pH module (REES52) directly with the Raspberry Pi. However, this connection proved infeasible: the Raspberry Pi does not have native analog input pins, and the pH sensor outputs an analog voltage signal that cannot be directly read without an external ADC. Attempts to bridge this using an ADC module were unsuccessful due to signal instability and interface compatibility issues. To ensure reliable analog sensor interfacing, the team transitioned to the Arduino UNO, which natively supports analog-to-digital conversion on pins A0–A5. This change enabled seamless and stable connection of both the pH sensor and the turbidity sensor.

B. Microcontroller and Components

The hardware prototype was implemented using the Arduino UNO (ATmega328P, 16 MHz). Real sensors were substituted for the simulated potentiometers. The REES52 analog pH sensor was connected to analog pin A0, and the JD23-2 turbidity sensor was connected to analog pin A1. The same tri-color LED and buzzer alert mechanism was retained.

C. Pin Configuration

Component	Arduino Pin
pH Sensor (REES52) AO	A0
Turbidity Sensor (JD23-2) AO	A1
Green LED	D8
Yellow LED	D9
Red LED	D10
Buzzer	D11

Table II. Arduino UNO Pin Mapping

D. Hardware Code

The following Arduino C++ code was deployed on the Arduino UNO hardware prototype:

Hardware Code (Arduino C++ — Arduino UNO)

```
// Arduino UNO Water Quality Monitoring System
// Without Temperature Sensor
// Connections:
// pH Sensor (REES52) AO/PO -> A0
// Turbidity Sensor (JD23-2) AO -> A1
// Green LED -> D8
// Yellow LED -> D9
// Red LED -> D10
// Buzzer -> D11

// -----
```

```
// Pin Definitions
// -----
const int phSensor = A0;
const int turbSensor = A1;
const int greenLED = 8;
const int yellowLED = 9;
const int redLED = 10;
const int buzzer = 11;

float ph, turb;

// -----
// Setup
// -----
void setup() {
  Serial.begin(9600);
  pinMode(greenLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(redLED, OUTPUT);
  pinMode(buzzer, OUTPUT);
}

// -----
// Read Sensor Values
// -----
void readValues() {
  int phValue = analogRead(phSensor);
  int turbValue = analogRead(turbSensor);
  ph = (phValue * 14.0) / 1023.0;
  turb = (turbValue * 20.0) / 1023.0;
}

// -----
// Analyze Water Condition
// -----
String analyze() {
  String causes = "";
  String corrections = "";
  String actions = "";
  String status = "";

  // pH Check
  if (ph < 6.5) {
    causes += "pH Low, ";
    corrections += "Increase pH by " + String(6.5 - ph, 2) + " | ";
    actions += "Add base (alkaline solution) | ";
  }
  else if (ph > 8.5) {
    causes += "pH High, ";
    corrections += "Decrease pH by " + String(ph - 8.5, 2) + " | ";
    actions += "Add acid | ";
  }

  // Turbidity Check
  if (turb >= 5) {
    causes += "High Turbidity, ";
    corrections += "Reduce Turbidity by " + String(turb - 5, 2) + " | ";
    actions += "Start filtration / sedimentation | ";
  }
}
```

```
// Decide Status
if (causes == "") {
    status = "SAFE";
}
else if ((ph >= 5 && ph <= 9) && turb < 10) {
    status = "WARNING";
}
else {
    status = "UNSAFE";
}

Serial.print("pH Value: ");
Serial.println(ph, 2);
Serial.print("Turbidity: ");
Serial.println(turb, 2);
Serial.print("Water Status: ");
Serial.println(status);

if (causes != "") {
    Serial.print("Cause: ");
    Serial.println(causes);
}
if (corrections != "") {
    Serial.print("Correction Needed: ");
    Serial.println(corrections);
}
if (actions != "") {
    Serial.print("Recovery Actions: ");
    Serial.println(actions);
}
Serial.println("-----");
return status;
}

// -----
// LED + Buzzer Control
// -----
void indicators(String status) {
    digitalWrite(greenLED, LOW);
    digitalWrite(yellowLED, LOW);
    digitalWrite(redLED, LOW);
    digitalWrite(buzzer, LOW);

    if (status == "SAFE") {
        digitalWrite(greenLED, HIGH);
    }
    else if (status == "WARNING") {
        digitalWrite(yellowLED, HIGH);
    }
    else {
        digitalWrite(redLED, HIGH);
        digitalWrite(buzzer, HIGH);
    }
}

// -----
// Main Loop
// -----
void loop() {
    readValues();
}
```

```
String status = analyze();
indicators(status);
delay(2000);
}
```

E. Hardware Prototype

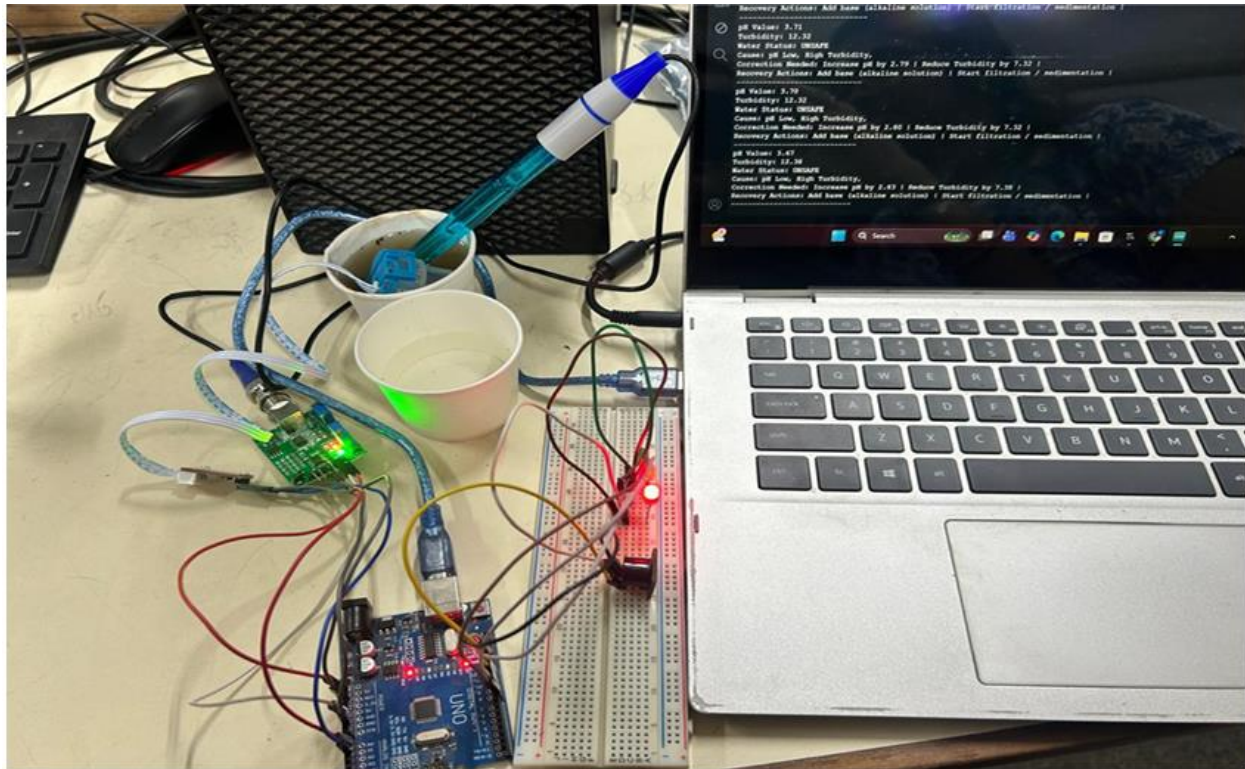


Fig. 2. Physical Hardware Prototype — Arduino UNO with pH and Turbidity Sensors on Breadboard

VI. System Classification and Alert Logic

The classification algorithm operates in three tiers:

- **SAFE:** All parameters within acceptable ranges. Green LED illuminated.
- **WARNING:** One or more parameters outside the ideal range but within recoverable bounds (pH 5–9; turbidity < 10 NTU). Yellow LED illuminated.
- **UNSAFE:** Parameters exceed critical thresholds. Red LED and buzzer activated simultaneously.

When anomalies are detected, the system generates specific corrective actions: recommending addition of alkaline or acidic solution for pH imbalance and initiating filtration or sedimentation for high turbidity. These suggestions are transmitted to the serial monitor for user review.

VII. Applications

The Smart Water Quality Monitoring System is broadly applicable across multiple domains:

- Residential and Municipal Water Supply: Continuous monitoring of drinking water pipelines to detect contamination events.
- Agricultural Irrigation: Ensuring irrigation water pH and turbidity are within crop-safe limits to avoid yield reduction.
- Aquaculture and Fish Farming: Real-time water quality management critical for maintaining aquatic life health.
- Industrial Effluent Monitoring: Pre-discharge verification of industrial wastewater to ensure regulatory compliance.
- Educational and Research Institutions: Serving as a low-cost learning platform for embedded systems and IoT courses.
- Remote Rural Communities: Providing affordable water safety monitoring where laboratory infrastructure is unavailable.

VIII. Comparative Advantages Over Conventional Low-Cost Checkers

Feature	Conventional Checker	Proposed System
Parameters Monitored	Usually 1 (e.g., pH only)	pH, Turbidity
Real-Time Continuous	No	Yes
Automated Alerts	None	LED + Buzzer
Corrective Guidance	None	Serial output suggestions
Multi-Status Classification	Single reading	SAFE / WARNING / UNSAFE
Expandability	Fixed	Modular, sensor-agnostic
Data Logging Potential	No	Yes (with IoT expansion)
Cost	Low	Comparable / Low

Table III. Comparison with Conventional Low-Cost Water Quality Checkers

IX. Key Features and Advantages

- Multi-Parameter Monitoring: Simultaneous measurement of pH and turbidity provides a comprehensive view of water quality.
- Three-Tier Classification: The SAFE / WARNING / UNSAFE classification enables proportional response and prevents alarm fatigue.
- Actionable Recovery Suggestions: Rather than displaying raw numbers, the system tells operators what corrective action is needed and by how much — a significant usability advantage.
- Dual-Platform Validation: Simulation-first development reduces hardware debugging time and validates logic before physical deployment.
- Low Cost and Accessible: Built on widely available Arduino UNO and common sensors, enabling deployment in resource-constrained environments.

- Scalable Architecture: The modular design allows additional sensors (dissolved oxygen, conductivity, temperature) to be integrated with minimal code changes.
- Immediate Alert Mechanism: The combination of visual LEDs and audible buzzer ensures alerts are perceivable even in noisy or low-light environments.

X. Future Enhancements

Several technology-driven enhancements are planned for future iterations of the system:

- IoT Cloud Connectivity: Integration of Wi-Fi modules (ESP8266 / ESP32) to transmit sensor data to cloud platforms such as AWS IoT, Google Firebase, or ThingSpeak for remote monitoring and historical trend analysis.
- Mobile Application Interface: Development of a companion mobile application to receive real-time alerts, view historical logs, and acknowledge warnings.
- Machine Learning-Based Prediction: Deployment of lightweight ML models (TinyML on Arduino Nano 33 BLE) to predict water quality degradation based on historical sensor patterns, enabling proactive intervention.
- Additional Sensor Integration: Incorporation of temperature, dissolved oxygen (DO), total dissolved solids (TDS), conductivity, and nitrate sensors to meet WHO multi-parameter water potability standards.
- Solar-Powered Deployment: Integration of photovoltaic power supply and battery management for off-grid remote deployment in agricultural fields and rural water bodies.
- GPS Geotagging: Adding GPS modules to tag contamination events with precise geographic coordinates for large-scale water body surveillance.
- OLED Display Integration: On-device display for standalone operation without requiring a connected computer for serial monitoring.
- Edge AI Anomaly Detection: Real-time anomaly detection at the sensor node level using edge computing to reduce cloud dependency and latency.

XI. Conclusion

This paper presented a Smart Water Quality Monitoring System that addresses the critical limitations of conventional water testing approaches. By integrating pH and turbidity sensors with the Arduino UNO microcontroller, and validating the design through a Wokwi-based Raspberry Pi Pico simulation, the system provides real-time, multi-parameter water quality assessment with automated three-tier classification, visual and audio alerting, and actionable corrective guidance.

The transition from Raspberry Pi Pico to Arduino UNO for hardware implementation was driven by the analog interfacing limitations of the Raspberry Pi, which lacks native ADC pins required for pH and turbidity sensor connectivity. The Arduino UNO resolved this compatibility issue and enabled stable, reliable sensor readings.

The proposed system demonstrates that embedded IoT platforms can deliver intelligent, continuous water monitoring at a fraction of the cost of laboratory testing or commercial multi-parameter analyzers. The dual-phase development approach — simulation followed by physical implementation — proved effective for validating system logic and accelerating hardware deployment.

With planned enhancements including IoT connectivity, machine learning-based prediction, and additional sensor modules, this system has significant potential for deployment in residential, agricultural, aquacultural, and industrial water management contexts. The work contributes to the broader goal of democratizing access to real-time environmental monitoring technology.

References

- [1] World Health Organization (WHO), "Guidelines for Drinking-water Quality," 4th ed., Geneva: WHO Press, 2011.
- [2] Bureau of Indian Standards, "IS 10500:2012 — Drinking Water Specification," New Delhi: BIS, 2012.
- [3] A. K. Srivastava, "IoT Based Water Quality Monitoring System," International Journal of Engineering Research and Technology, vol. 9, no. 5, pp. 234–238, 2020.
- [4] Raspberry Pi Foundation, "Raspberry Pi Pico Datasheet," 2021. [Online]. Available: <https://datasheets.raspberrypi.com/pico/pico-datasheet.pdf>
- [5] Arduino, "Arduino UNO Rev3 Datasheet," 2023. [Online]. Available: <https://store.arduino.cc/products/arduino-uno-rev3>
- [6] Wokwi, "Wokwi Online Embedded Systems Simulator," 2023. [Online]. Available: <https://wokwi.com>
- [7] M. A. H. Akhand et al., "Real-Time Water Quality Monitoring Using IoT and Embedded Systems," IEEE Access, vol. 9, pp. 12345–12357, 2021.
- [8] N. Rajagopal and K. Ramachandran, "Low-Cost pH and Turbidity Monitoring System for Drinking Water Using Arduino," in Proc. IEEE International Conference on Smart Systems and Inventive Technology (ICSSIT), 2020, pp. 456–460.